Expert Systems with Applications 38 (2011) 235–248

Contents lists available at ScienceDirect

Expert Systems with Applications

journal homepage: www.elsevier.com/locate/eswa

A hybrid particle swarm optimization for a university course scheduling problem with flexible preferences

Der-Fang Shiau *

Department of Information Management, Fooyin University, Kaohsiung, Taiwan, ROC

ARTICLE INFO

Keywords:
Course scheduling
Particle swarm optimization
Flexible preferences
Local search
Repair process

ABSTRACT

The timetabling problem at universities is an NP-hard problem concerned with instructor assignments and class scheduling under multiple constraints and limited resources. A novel meta-heuristic algorithm that is based on the principles of particle swarm optimization (PSO) is proposed for course scheduling problem. The algorithm includes some features: designing an 'absolute position value' representation for the particle; allowing instructors that they are willing to lecture based on flexible preferences, such as their preferred days and time periods, the maximum number of teaching-free time periods and the lecturing format (consecutive time periods or separated into different time periods); and employing a repair process for all infeasible timetables. Furthermore, in the original PSO algorithm, particles search solutions in a continuous solution space. Since the solution space of the course scheduling problem is discrete, a local search mechanism is incorporated into the proposed PSO in order to explore a better solution improvement. The algorithms were tested using the timetabling data from a typical university in Taiwan. The experimental results demonstrate that the proposed hybrid algorithm yields an efficient solution with an optimal satisfaction of course scheduling for instructors and class scheduling arrangements. This hybrid algorithm also outperforms the genetic algorithm proposed in the literature.

© 2010 Elsevier Ltd. All rights reserved.

1. Introduction

Scheduling problems are concerted with how to optimally assign the limited resources to tasks over time. Most scheduling problems are categorized as NP complete or combinatorial problems, such as communications, industrial control, operational research and production planning. This study focuses on a special class of scheduling problems, known as the university course scheduling problem (UCSP) and has been proved to be an NP-hard (Bardadym, 1996). The UCSP is simple to understand, yet complex enough to admit a wide range of solutions at varying levels of difficulty in implementation (Combs et al., 2005). The most important tasks of UCSP involve various constraints, such as instructors' preferences, curriculum planning policies of school, student scheduling, classroom assignment and available teaching resources. Moreover, other constraints such as administrative rules and regulations of institutions, administration's coordination, the cost consideration and individual expectation also constitute the additional constraints of the course scheduling. This fact implies that an optimal or near-optimal solution for a large scheduling problem can be quite time-consuming. Therefore, it is necessary to adopt efficient search algorithms to generate optimal or near-optimal solutions that satisfy those constraints.

A large variety of approaches have been applied to course scheduling problems, such as genetic algorithms (GA) and simulated annealing (SA) (Merlot, Boland, Hughes, & Stuckey, 2002; Valdes, Crespo, & Tamarit, 2002; Wang, 2002). Because of the intractable nature of the course scheduling problem, it is desirable to explore other avenues for developing good algorithms for UCSP. The emphasis of this study proposes a novel meta-heuristic method, named particle swarm optimization (PSO) to solve UCSP.

The particle swarm optimization (PSO) is first proposed by Kennedy and Eberhart (1995). In PSO, a swarm of particles spread in the space and the position of a particle presents a solution. Each particle would move to a new position decided with the global experience and the individual experience heading for the global optimum. Previous study (Lo, Chen, Shiau, & Wu, 2008) demonstrated that the approach is able to obtain more optima or near-optima solutions for the cases of resource-constrained scheduling problems. Luo, Wang, Tang, and Tu (2006) used PSO to solve the resource-constrained project scheduling problem; they showed that PSO is applicable to various combinatorial and scheduling problems. Furthermore, Salman, Ahmad, and Al-Madani (2002) presented PSO for task assignment problem; they also demonstrated that the proposed PSO-based algorithm solution quality is better than that of genetic algorithm (GA) in most of cases. Moreover, the PSO algorithm runs faster as compared with GA. Tseng and Liao (2008) demonstrated that the proposed PSO algorithm outperforms over GA and ant colony optimization (ACO) for the flow shop

* Tel.: +886 7 7811151x5721; fax: +886 7 7821301.
E-mail address: ft041@mail.fy.edu.tw

0957-4174/$ - see front matter © 2010 Elsevier Ltd. All rights reserved.
doi:10.1016/j.eswa.2010.06.051

scheduling problem. More recently, other applications of PSO for scheduling problem can be found in Kuo et al. (2009) and Sha and Hsu (2006). This study introduces an approach based on PSO for UCSP. The proposed hybrid PSO (HPSO) algorithm contains several features. The first is to design an 'absolute position value' representation for the particle. The second is to allow instructors that they are willing to teach based on flexible preferences, such as their preferred days and time periods, the maximum number of teaching-free time periods and the lecturing format (consecutive time periods or separated into different time periods). The third, a repair process is involved, which ensures that all infeasible timetables are rectified. Furthermore, since the solution space of UCSP is discrete, and a local search scheme is incorporated to explore a better solution improvement. The experimental results demonstrate that the hybrid approach yields an optimal satisfaction of course scheduling for instructors and class scheduling arrangements.

The remainder of this study is organized as follows. In Sections 2 and 3, a survey of literature and the university course scheduling problem are discussed, respectively. The structure of HPSO is presented in Section 4. In Section 5, experimental results and discussion are made. Finally, some conclusions follow in Section 6.

2. Literature review

The university course scheduling problem (UCSP) is one of the scheduling problems that has been extensively studied over the last 25 years (Valdes et al., 2002). Many studies solve UCSP using the meta-heuristics based methods, such as simulated annealing (Pongcharoen, Promtet, Yenradee, & Hicks, 2008; Thompson & Dowsland, 1998), genetic algorithms (Drexl & Salewski, 1997; Pongcharoen et al., 2008; Wang, 2002, 2003), tabu search (TS) (Alvarez-Valdes, Crespo, & Tamarit, 2002) and ant colony optimization (Azimi, 2005). Azimi (2005) employed hybrid heuristics involved four meta-heuristics approaches (GA, SA, TS and ACO) for examination timetabling problem. The results showed that ACO produced the best solution, followed by TS and GA and the worst was SA. Nevertheless, Pongcharoen et al. (2008) demonstrated that the results of both GA and SA produced good timetables, but the results obtained from SA were slightly better than the GA. However, the GA was faster than SA in computation time.

Some effective hybridization of exact algorithms and meta-heuristics has been shown to achieve better quality, time performance, and robustness than pure algorithm methods in scheduling problem. Shiau, Cheng, and Huang (2008) combined GA and tabu search for solving flow shop scheduling. They showed that the evolutionary process to a population trained by a local improvement search bases on tabu search. One of the earlier studies for UCSP using the idea of hybrid algorithms was proposed by Weare, Burke, and Elliman (1995). They adopted a genetic algorithm that combines a direct representation of the timetable and heuristic crossover operators. Some other applications using hybrid algorithm can be found in Chiarandini and Stützle (2003), Duong and Lam (2004) and Merlot et al. (2002). Regarding to the latest applications of the hybrid algorithm for UCSP was employed by Chiarandini, Birattari, Socha, and Rossi-Doria (2006). Gunawan et al. (2007) proposed a hybrid algorithm that combines an integer programming, a greedy heuristic and a modified SA for solving UCSP. According to the research work by Wang (2005), a combination of different heuristics and a combination of heuristics and AI (neural networks, GAs, TS and SA) search techniques can greatly improve the quality of the solutions and also be a future trend of algorithm development.

To conclude the characteristics of above stated meta-heuristic algorithms, most of them require a fitness function to evaluate the quality and feasibility of solution. The feature of a good solution is referred when constructs a new solution. Moreover, a fragment of the good solution is combined with another fragment of another feasible solution to build a new solution in GA. The heuristic methods introduced in these algorithms which infers new solution from the helpful experience, and is possible to produce premature convergence. The local optimal solution will incur in these algorithms. Therefore, most algorithms usually include additional techniques to escape from the premature convergence. For example, randomly disturbing the system or depressing the self-reinforce is applied (Zhang, Li, & Tam, 2006). Eventually, there are two aspects to improve the algorithm for scheduling global optimal solution; one is to enhance the heuristic ability (exploitation ability), the other is to strengthen the ability of escaping local optimum or extending the search space (exploration ability).

3. Problem description and assumptions for UCSP

Marriott and Stuckey (1998) showed that the problem of generating course schedules is a classic example of a constraint satisfaction problem. The UCSP has arisen in the context of a university in Taiwan. It consists of a set of courses to be scheduled in 40 timeslots across five days and eight periods. At the beginning of new semester, several courses are offered to students (classes) and each course can be divided into various sections according to the number of students enrolled. The UCSP problems are categorized into two types of constraints: hard constraints and soft constraints. Hard constraints must be satisfied in order to generate feasible solutions. Soft constraints are sometimes referred as preferences that can be contravened if necessary. That is, schedule generation can be optimized by finding solutions that violate the minimal number of preferences for instructors and classes. It is usually difficult or impossible to generate course schedules that satisfy all instructors' preferences (Burke & Petrovic, 2002). Moreover, there are 14 hard constraints and 18 soft constraints that have been considered by previous research and also can be found in Pongcharoen et al. (2008). The hard constraints in this study are described as follows:

(1) Each instructor can only teach one course section at a time.
(2) Each course section can only be taught by an instructor.
(3) Students can only attend one course section at a time.
(4) Each classroom can not be used more than one course section at a time period.
(5) Certain time periods can not be scheduled for non-academic activities, such as lunch time and sport.
(6) Some course sections have to be scheduled in a particular room, such as computer lab.

In addition, some other soft constraints to be considered:

(1) Instructors and students (classes) can indicate their preferences, along with their preferred days and time periods in 40 timeslots across five days and eight periods.
(2) Instructors can choose to maximize the number of teaching-free time periods; i.e., instructors can specify time periods when they prefer not to lecture.
(3) A course section within three teaching hours has to be scheduled at the consecutive time periods and can not be separated into two days of different time periods if required; i.e., instructors can specify their preferred lecturing format for a specific course section (for example, a course section requires three teaching periods, the formats of 3-0, 2-1 and 1-1-1 represent three consecutive periods, two consecutive periods with a further single period and three single periods, respectively).
(4) Minimizing students' movement between rooms.

PERIOD	Instructor 1						Instructor m				
	Mon	Tue	Wed	Thu	Fri	…	Mon	Tue	Wed	Thu	Fri
1	0	0	0	-1	5	…	0	5	4	-1	5
2	1	3	4	-1	5	…	2	5	4	2	5
3	1	3	4	-1	5	…	2	5	4	2	5
4	1	3	4	-1	5	…	2	5	4	2	5
5	3	5	-1	3	4	…	1	3	-1	1	-1
6	3	5	-1	3	4	…	1	3	-1	1	-1
7	3	5	-1	3	-1	…	1	2	-1	1	-1
8	0	0	-1	3	-1	…	0	2	-1	1	-1

Fig. 1. The priority values of preferences for instructors.

It is worth to mention that none of previous studies address the four soft constraints into their algorithms simultaneously. In particular, the flexibilities of soft constraints 1-3 are considered to be important for many academics and are also incorporated in this study. Some variables are used in this study, let $T = \{T_1, T_2, \ldots, T_m\}$ is the set of m instructors to be scheduled, $C = \{C_1, C_2, \ldots, C_n\}$ represents the set of n-classes to be scheduled and $S = \{S_1, S_2, \ldots, S_p\}$ be a set of p course sections offered in a certain department. This study assumes that each instructor's willingness to each various course sections is known. Let L_t be the subset of S that represents the number of course sections taught by instructor t, and H_k be the subset of S that indicates the number of course sections to be assigned to class k. Thus, each instructor assigns the priority values based on their preferences, along with their preferred days and time periods (soft constraints 1). Fig. 1 represents the example of priority value of each time period in the range 0–5 for each instructor. The number '5' is indicated the highest priority to the specific time period in teaching a particular course section, while the number '−1' is specified to the time period when instructors prefer not to lecture (soft constraints 2); i.e., instructors can choose to maximize the number of teaching-free time periods for doing other work. Furthermore, the priority values assigned by each class are the same as the setting of instructors. The overall objective is to find a feasible timetable that satisfies all the hard constraints to maximize the total priority values (TPV) for each of instructor and class, which is the weighted sum of priority values. This function is described as follows:

$$\text{MAX}(\text{TPV})$$

$$\text{TPV} = \alpha\left[\sum_{t\in T}\sum_{l\in L_t}\text{PV}_{lt}\right] + \beta\left[\sum_{k\in C}\sum_{h\in H_k}\text{PV}_{hk}\right] \tag{1}$$

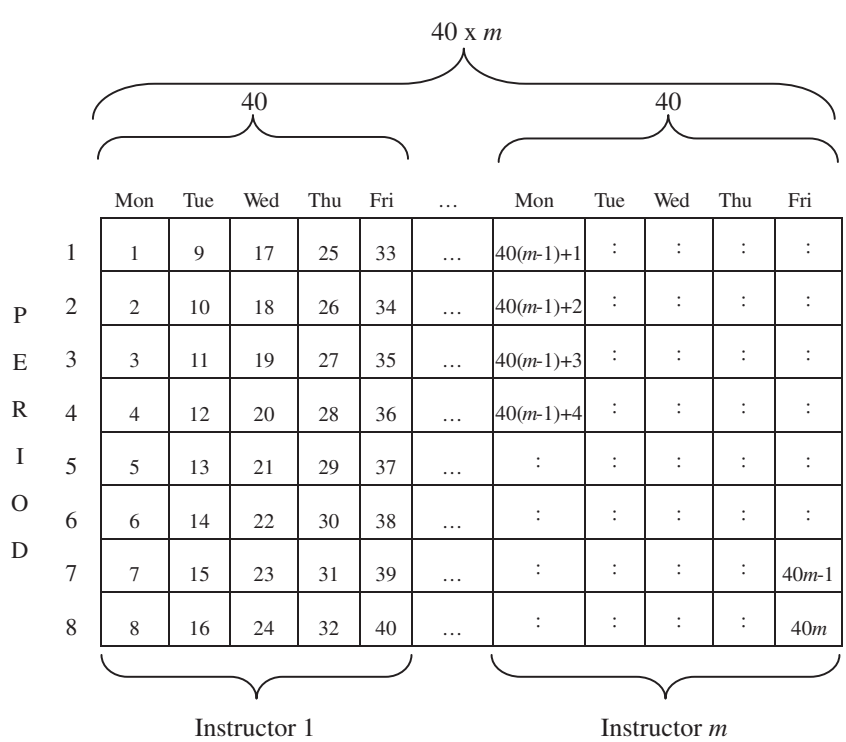

Fig. 2. The representation of course scheduling.

where PV_{lt} indicates the sum of priority values for those of time periods in teaching course section l for instructor t ($t \in T$) and PV_{hk} represents the sum of priority values for those of time periods in offering course section h for class k ($k \in C$). In other words, the objective is to obtain an optimal satisfaction for each of instructor and class based on their priority values of preferences. In addition, according to the priority values shown in Fig. 1, a course section allocates to the time periods indicated by the priority value '−1', which violates the soft constraint 2. This means that the objective (fitness) function applies a penalty to the time periods, each of which specifies a non-positive value indicated by the number '−1'. Furthermore, two weighting factors α and β specify the relative priorities to be given the degree of satisfaction for instructors and classes in the optimization process. Restated, the degree of satisfaction for instructors can be emphasized by specifying a high value. α plus β equals one.

4. Hybrid particle swarm optimization (HPSO) and UCSP

The particle swarm optimization (PSO) was designed, developed and tested to solve UCSP. Before presenting the algorithm, the PSO meta-heuristic is introduced as follows.

4.1. Background of PSO

The PSO algorithm is a multi-agent general meta-heuristic method, and can be applied extensively in solving many difficult problems (Kennedy & Eberhart, 1995). The PSO consists of a swarm of particles in the space; the position of a particle is indicated by a vector which presents a solution. PSO is initialized with a population of M random particles and searches for the best position (solution or schedule). In every generation or iteration, the local best and global best are determined through evaluating the performances in terms of the fitness values of current population of particles. A particle moves to a new position obtaining a new solution guided by the velocity (a vector). Hence, the velocity plays an important role in affecting the characters of creating new solution. There are two experience positions used in the PSO; one is the global experience position of all particles, which memorizes the global best solution obtained from all positions (solutions) of all particles; the other is the individual experience position of each particle, which memorizes the local best solution acquired from the positions (solutions) of the corresponding particle has been at. These two experience positions and the inertia weight of the previous velocities are used to determine the impact on the current velocity. The velocity retains part of prior velocity (the inertia) and drives particle toward the direction based on the global experience position and the individual experience position. Thus, the particles can derive new positions (solutions) by their own inertia and experience positions.

Let the search space is D dimension space (the number of dimension is corresponding to the parameters of solutions) and the population consists of M particles. $X_i = \{X_{i1}, \ldots, X_{iD}\}$, X_{iD} be the particle i with D dimension space ($i = 1, \ldots, M$). A position X_i has a rate of position change called velocity $V_i = \{V_{i1}, \ldots, V_{iD}\}$. The individual experience is a position $L_i = \{L_{i1}, \ldots, L_{iD}\}$, the local best position for the ith particle (called pbest) . Additionally, $G = \{G_1, \ldots, G_D\}$ represents the global best position among all the population of particles achieved so far (called gbest). The PSO algorithm could be performed by the Eq. (2)

$$V_{id}^{new} = wV_{id} + c_1 r_1 (L_{id} - X_{id}) + c_2 r_2 (G_d - X_{id}) \quad (2)$$

$$X_{id}^{new} = X_{id} + V_{id}^{new} \quad (3)$$

where w is an inertia weight used to determine the influence of the previous velocity to the new velocity. The c_1 and c_2 are learning factors used to derive how the ith particle approaching the position closes to the individual experience position or global experience position, respectively. Furthermore, the r_1 and r_2 are the random numbers uniformly distributed in [0, 1], influencing the tradeoff

Table 1
The example of the schedule for instructor t.

S_k	Classroom	Days	Period	Class
CS1	PC1	Mon	2, 3 and 4	A
SE2	PC1	Tue	2, 3 and 4	B
SC3	PC1	Wed	2, 3 and 4	B

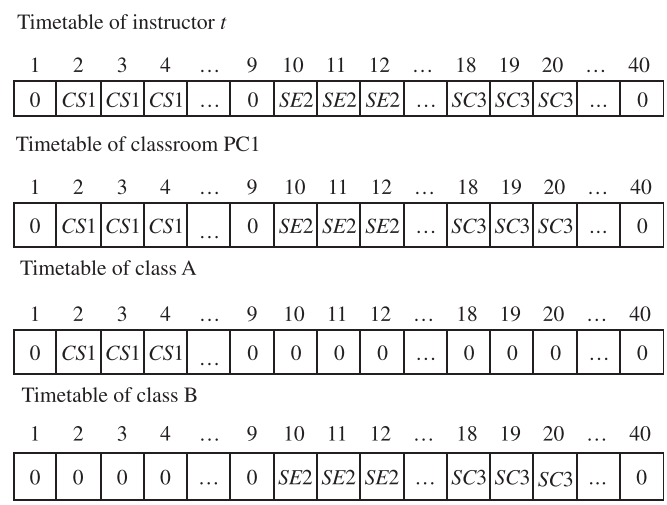

Fig. 3. The example of the instructor's timetable to the room and class timetable mapping.

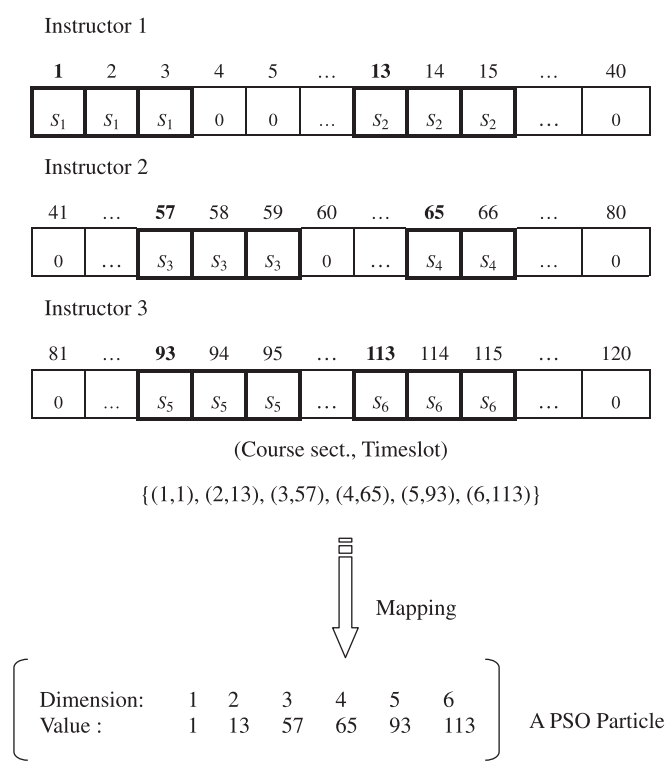

Fig. 4. Course scheduling to PSO particle mapping.

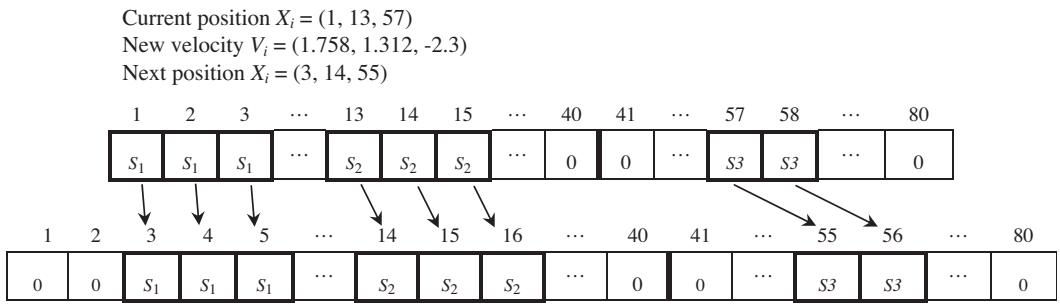

Fig. 5. Update of current particle position to a new position.

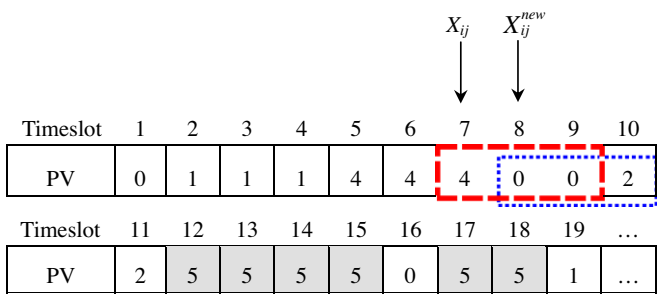

Fig. 6. An example of non-consecutive priority values.

between the global and local exploration abilities during search. The description of a PSO algorithm is given as follows:

1. **Initialization:** Initialize a population of particles with random positions and velocities on D dimensions in the search space.
2. **Evaluation:** Evaluate the fitness of each particle in the population.
3. **Update:** Calculate the velocity of each particle by Eq. (2) and move to the next position according to Eq. (3).
4. **Termination:** Stop the algorithm if a specified stopping criterion is reached; go to step 2, otherwise.

4.2. Proposed HPSO algorithm for UCSP

This subsection begins to develop a HPSO algorithm for UCSP. The proposed algorithm includes several features, such as associating a particle position into the timetable of instructors and classes, allowing instructors that they are willing to lecture based on their preferences, and employing a repair process for all infeasible timetables. In addition, since the solution space of the UCSP is discrete, and a local search mechanism is incorporated into the PSO algorithm to explore the particle to a better position.

4.2.1. Encoding of particles

The most important issue in applying PSO to UCSP is to define an encoding scheme that allows one-to-one mapping between solution and particle (i.e., the particles search solutions in a continuous solution space). Each particle i implicitly consists of timetables for instructors, classes and classrooms. For an m-instructor n-class problem, this study represents the particle i position by a search space of one dimensional matrix, as shown in Fig. 2. That is, eight teaching periods of classes are offered for each day, and students study five working days a week; thus, each continuous 40 timeslots across five working days and eight periods for each instructor and class. Timeslots 1–40 represent the timetable of the first instructor; timeslots 41–80 represent the timetable of the second instructor, and so on.

Example 1. Suppose that an instructor t teaches three course sections for classes A and B in the certain semester. The result of a weekly schedule for instructor t as shown in Table 1. Fig. 3 indicates a mapping between the timetable of instructor t and the timetables of classroom and classes.

The proposed algorithm designs an 'absolute position value' representation for the particle. Since each instructor's willingness to lecture various course sections is known. Therefore, this study represents the position of particle i as an p-dimensional vector ($p \in S$), whose entries' values belong to the set $(1, 2, \ldots, 40 \times m)$; i.e., a particle moves in an p-dimensional search space. The position of particle i is defined as $X_i = (X_{i1}, X_{i2}, \ldots, X_{ip})$, where X_{ij} represents

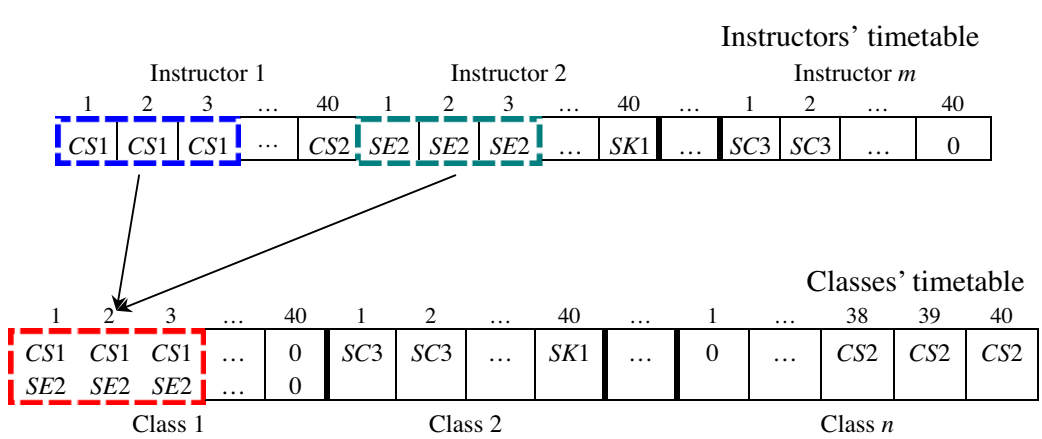

Fig. 7. The repair process for an infeasible timetable that violates hard constraint 3.

the starting timeslot of course section j for particle i. For example, consider the problem to a 3-instructor course scheduling. Fig. 4 shows a mapping between a course scheduling instance to a particle position coordinates in the PSO domain. The first course section S_1 starts from the first timeslot (the first time period on Monday for the first instructor), the second course section S_2 starts from the 13th timeslot (the fifth time period on Tuesday for the first instructor) and the third course section S_3 starts from the 57th timeslot (the first time period on Wednesday for the second instructor), and so on. Thus, $X_i = (X_{i1}, X_{i2}, \ldots, X_{i6}) = (1, 13, 57, 65, 93, 113)$. As mentioned in Section 3, instructors can specify their preferred lecturing format for a specific course (soft constraint 3). For example, if there is a course section s_k with three teaching periods, the instructor may prefer two consecutive periods long together with

Initialization:

1. Initialize parameters w, c_1 and c_2.
2. For each particle i in the population
   - For each course section j:
     - Initialize $PSO[i][j]$ randomly: initialize the timetable of each instructor that satisfies all hard constraints described in Section 3.
   - Evaluate $fitness[i]$.
   - Initialize $V[i]$ randomly.
   - Initialize the local best position $pbest[i]$.
   - Initialize G_{best} with the best fitness value among the population.

Process of HPSO:

(1) ***while*** (a number of generation is not reach) ***do***
(2) *Find* G_{best} *with* $fitness[G_{best}] \geq fitness[i] \quad \forall i \leq P$
(3) ***for*** *each particle i* ***do***
(4) ***if*** $fitness[i] > pbest_fitness[i] \quad \forall i \leq P$ ***then***
(5) $pbest[i] \leftarrow PSO[i]$
(6) ***end if***
(7) ***for*** *each course section j* ***do***
(8) *Compute new velocity* V_{ij}^{new} *according to Eq.* (2)
(9) *Compute new position* X_{ij}^{new} *according to* $X_{ij}^{new} \leftarrow X_{ij} + V_{ij}^{new}$ (*Eq.* 3)
(10) ***If*** X_{ij}^{new} *violates the one of hard constraints* ***then***
(11) *Apply repair process*
(12) ***else***
(13) ***if*** *priority values of* X_{ij}^{new} < *priority values of* X_{ij} ***then***
// Local search
(14) $X_{ij}^{*,new} \leftarrow X_{ij} + V_{ij}^{new} + Rand(-L_{max}, +L_{max})$
(15) ***if*** *priority values of* $X_{ij}^{*,new}$ > *priority values of* X_{ij} ***then***
(16) ***if*** $X_{ij}^{*,new}$ *satisfies all hard constraints* ***then***
(17) *Accept the new position* $X_{ij}^{*,new}$
$X_{ij}^{new} \leftarrow X_{ij}^{*,new}$
(18) ***Else***
(19) $X_{ij}^{new} \leftarrow X_{ij} + V_{ij}^{new}$
(20) ***end if***
(21) ***end if***
(22) ***end if***
(23) ***end if***
(24) ***end for***
(25) *Update* $V[i]$ and $PSO[i]$
(26) *Caculate* $fitness[i] \quad \forall i \leq P$
(27) ***end for***
(28) ***end while***
return $PSO[G_{best}]$

Fig. 8. Pseudo code of HPSO process for UCSP.

a further single period instead of three consecutive time periods. This can be easily solved by dividing the course section s_k into two distinct subsections with 2-1 format, each of which represents one position vector in X_i.

4.2.2. Fitness of particles

The next stage of the HPSO is to measure the quality of timetables (particle). As mentioned in Section 3, since each instructor's willingness to teach various courses is known; and therefore each instructor assigns the priority values based on their preferences. According to Fig. 1, the overall objective is to find the feasible timetables that satisfy all the hard constraints to maximize the total priority values (TPV). In other words, the objective is to obtain an optimal satisfaction for each of instructor and class based on their priority values of preferences. Then, the fitness function fitness [i] be the objective function of ith particle according to Eq. (1); i.e., fitness[i] = TPV.

4.2.3. Update of velocity

For the course scheduling problem, the algorithm uses the new velocity based on Eq. (2) to update the particle current position to a new position according to Eq. (3). This study defines the velocity of particle i as $V_i = (V_{i1}, V_{i2}, \ldots, V_{ip})$, $V_{ij} \in \mathrm{R}$, where V_{ij} is the movement of course section j for particle i from current timeslot to the next one. For example, velocities vector, $V_i = (1.758, 1.312, -2.3)$, in the next generation is added to position vector, $X_i = (1, 13, 57)$, in previous generation to give a new position vector equals to (2.758, 14.312, 54.70). Since values in particle are timeslot numbers, a real value such as 2.758 is meaningless. Therefore this study always uses the integer position values to round up these numbers to the timeslot number. Thus, the particle position as (3, 14, 55) is obtained in the next generation. Fig. 5 shows the update of current particle position to a new position. Generally, the value of each component in V can be clamped to the range $[-V_{max}, +V_{max}]$ to control excessive roaming of particles outside the search area. This study sets the V_{max} value of 3, which limit to move forward or backward at most 3 timeslots for each course section j. That is, if V_{ij} is smaller than $-V_{max}$, then set $V_{ij} = -V_{max}$; if V_{ij} is greater than V_{max}, then set $V_{ij} = V_{max}$. Furthermore, if an instructor had specified a lecturing format associated with soft constraint 3. If the new position violates this soft constraint, then the fitness function applies a penalty. The fitness function of ith particle with penalty is shown as follows:

Table 2
Course sections offered of each class.

Class IM1A			Class IM1B			Class IM2A			Class IM2B		
Course sect. no.	Hrs.	Instructor no.	Course sect. no.	Hrs.	Instructor no.	Course sect. no.	Hrs.	Instructor no.	Course sect. no.	Hrs.	Instructor no.
A001-1A	3	TA03	A001-1B	3	TA03	A002-2A	3	TA01	A002-2B	3	TA01
A005-1A	3	TA06	A005-1B	3	TA06	A012-2A	3	TA11	A012-2B	3	TA16
A007-1A	3	TA02	A007-1B	3	TA02	A020-2A	3	TA13	A020-2B	3	TA07
A008-1A	3	TA05	A008-1B	3	TA05	A033-2A	3	TA10	A033-2B	3	TA10
D019-1A	3	TA16	D019-1B	3	TA13	D011-2A	3	TA10	D030-2B	3	TA08
D020-1A	3	TA16	D020-1B	3	TA13	T009-2A	3	TA04	T009-2B	3	TA09
Class IM3A			Class IM3B			Class IM4A			Class IM4B		
A013-3A	3	TA04	A013-3B	3	TA04	L017-4A	3	TA08	D013-4B	3	TA07
A022-3A	3	TA03	A022-3B	3	TA01	L020-4A	3	TA09	L015-4B	3	TA08
C008-3A	3	TA12	C008-3B	3	TA12	L021-4A	3	TA07	L018-4B	3	TA02
C011-3A	3	TA11	C011-3B	3	TA11	S017-4A	3	TA17	L020-4B	3	TA09
T010-3A	3	TA14	T010-3B	3	TA14	S022-4A	3	TA05	S022-4B	3	TA12
S020-3A	3	TA17	S020-3B	3	TA17	T031-4A	3	TA15	T031-4B	3	TA15

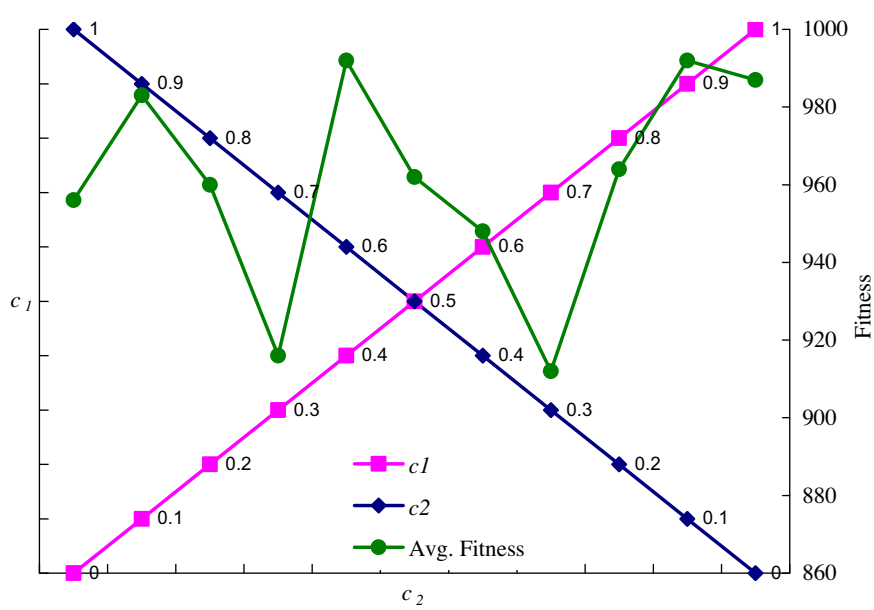

Fig. 9. The preliminary experiment for various combinations of c_1 and c_2.

Table 3
The results of course scheduling for each instructor.

Instructor no.	Course sect. no.	Class	Class room	Days	Period
TA01	A002-2A	IM2A	T1303	Wed	2–3–4
	A002-2B	IM2B	T1305	Fri	1–2–3
	A022-3B	IM3B	T4203	Thu	1–2–3
TA02	A007-1B	IM1B	T0202	Wed	2–3–4
	A007-1A	IM1A	T0201	Mon	1–2–3
	L018-4B	IM4B	T4201	Fri	2–3–4
TA03	A022-3A	IM3A	T4201	Wed	2–3–4
	A001-1B	IM1B	T0202	Fri	2–3–4
	A001-1A	IM1A	T0201	Thu	2–3–4
TA04	A013-3B	IM3B	T4203	Mon	5–6–7
	A013-3A	IM3A	T4201	Fri	5–6–7
	T009-2A	IM2A	T1303	Thu	5–6–7
TA05	A008-1B	IM1B	T0202	Fri	6–7–8
	A008-1A	IM1A	T0201	Thu	6–7–8
	S022-4A	IM4A	T4302	Tue	6–7–8
TA06	A005-1B	IM1B	T0202	Mon	2–3–4
	A005-1A	IM1A	T0201	Wed	2–3–4
TA07	A020-2B	IM2B	T3402	Tue	6–7–8
	L021-4A	IM4A	T4203	Thu	5–6–7
	D013-4B	IM4B	T3502	Fri	6–7–8
TA08	D030-2B	IM2B	T4302	Mon	5–6–7
	L015-4B	IM4B	T4203	Tue	1–2–3
	L017-4A	IM4A	T3402	Wed	2–3–4
TA09	T009-2B	IM2B	T1305	Fri	6–7–8
	L020-4A	IM4A	T3502	Tue	1–2–3
	L020-4B	IM4B	T4302	Wed	2–3–4
TA10	A033-2A	IM2A	T1303	Fri	2–3–4
	A033-2B	IM2B	T1305	Wed	2–3–4
	D011-2A	IM2A	T3402	Thu	2–3–4
TA11	A012-2A	IM2A	T1303	Tue	2–3–4
	C011-3A	IM3A	T3502	Mon	5–6–7
	C011-3B	IM3B	T4302	Fri	2–3–4
TA12	S022-4B	IM4B	T3402	Thu	6–7–8
	C008-3A	IM3A	T4201	Mon	2–3–4
	C008-3B	IM3B	T4203	Wed	2–3–4
TA13	A020-2A	IM2A	T3502	Tue	5–6–7
	D019-1B	IM1B	T0202	Tue	2–3–4
	D020-1B	IM1B	T3502	Thu	2–3–4
TA14	T010-3A	IM3A	T2202	Thu	1–2–3
	T010-3B	IM3B	T2202	Mon	1–2–3
TA15	T031-4A	IM4A	T2202	Fri	6–7–8
	T031-4B	IM4B	T2202	Tue	5–6–7
TA16	D019-1A	IM1A	T0201	Mon	6–7–8
	A012-2B	IM2B	T1305	Thu	1–2–3
	D020-1A	IM1A	T4302	Fri	5–6–7
TA17	S020-3A	IM3A	T3402	Tue	1–2–3
	S020-3B	IM3B	T3502	Thu	5–6–7
	S017-4A	IM4A	T4302	Thu	1–2–3

$$\text{fitness}[i] = \text{TPV} - \sum_{h=1}^{p} \text{pen}_{(i,h)} \quad (h \in S) \tag{4}$$

where $\text{pen}_{(i,h)}$ denotes the penalty value of course section h for particle i that violates the soft constraint 3. The different penalty values can be used to specify the relative priorities for each course section. This means that a course section can be emphasized by giving a high penalty value. This study used equal penalty value for each course section.

4.2.4. Local search for the UCSP

When a particle moves in a continuous solution space, due to inertia, the particle velocity moves the particle to a better position. Since the solution space of the UCSP is discrete; i.e., the priority values are non-consecutive for each of instructor, as described in Fig. 1. In the example shown in Fig. 6, the course section j with three teaching periods is assigned to the timeslots of 7, 8 and 9 ($X_{ij} = 7$) and the corresponding priority values (PV) are 4, 0 and 0 (the sum of PV is 4). According to the PSO algorithm, if the course section j moves to the next position in the next generation (say $X_{ij}^{new} = 8$), the sum of PV is 2; possibly will no longer move backward in further generations. However, some higher priority values may exist in the following positions.

For the above descriptions, this study incorporates a local search mechanism into the proposed PSO, and modifies the particle position by rewriting the Eq. (3) as follows:

$$X_{id}^{new} = X_{id} + V_{id}^{new} + Rand(-L_{max}, +L_{max}) \tag{5}$$

where $Rand$ denotes a random integer number between $-L_{max}$ and $+L_{max}$. After a new position X_{ij}^{new} is obtained according to Eq. (3), the process then checks the neighborhoods of the new position X_{ij}^{new} whether there is a better position (generated by executing $Rand(-L_{max}, +L_{max})$) with a higher PV than the new position Xij^{new}. If there is a such position existed, then the new position Xij^{new} is updated based on Eq. (5), otherwise, the new position is kept unchanged.

4.2.5. Repair process

The algorithm starts to generate randomly as many potential solutions for the problem as the size of the initial population of the PSO. The each dimensional vector in the particle represents a course section taught by a specified instructor. The allocation of each course section is performed sequentially until all course sections are assigned to the available time periods. Each solution

Table 4
The results of course scheduling for each class.

Classes	Course sect. no.	Instructor no.	Class room	Days	Period
IM1A	A007-1A	TA02	T0201	Mon	1–2–3
	D019-1A	TA16	T0201	Mon	6–7–8
	A005-1A	TA06	T0201	Wed	2–3–4
	A001-1A	TA03	T0201	Thu	2–3–4
	A008-1A	TA05	T0201	Thu	6–7–8
	D020-1A	TA16	T4302	Fri	5–6–7
IM1B	A005-1B	TA06	T0202	Mon	2–3–4
	D019-1B	TA13	T0202	Tue	2–3–4
	A007-1B	TA02	T0202	Wed	2–3–4
	D020-1B	TA13	T3502	Thu	2–3–4
	A001-1B	TA03	T0202	Fri	2–3–4
	A008-1B	TA05	T0202	Fri	6–7–8
IM2A	A002-2A	TA01	T1303	Wed	2–3–4
	A020-2A	TA13	T3502	Tue	5–6–7
	A012-2A	TA11	T1303	Tue	2–3–4
	A033-2A	TA10	T1303	Fri	2–3–4
	T009-2A	TA04	T1303	Thu	5–6–7
	D011-2A	TA10	T3402	Thu	2–3–4
IM2B	D030-2B	TA08	T4302	Mon	5–6–7
	A020-2B	TA07	T3402	Tue	6–7–8
	A033-2B	TA10	T1305	Wed	2–3–4
	A012-2B	TA16	T1305	Thu	1–2–3
	A002-2B	TA01	T1305	Fri	1–2–3
	T009-2B	TA09	T1305	Fri	6–7–8
IM3A	C008-3A	TA12	T4201	Mon	2–3–4
	C011-3A	TA11	T3502	Mon	5–6–7
	S020-3A	TA17	T3402	Tue	1–2–3
	A022-3A	TA03	T4201	Wed	2–3–4
	T010-3A	TA14	T2202	Thu	1–2–3
	A013-3A	TA04	T4201	Fri	5–6–7
IM3B	A013-3B	TA04	T4203	Mon	5–6–7
	T010-3B	TA14	T2202	Mon	1–2–3
	C008-3B	TA12	T4203	Wed	2–3–4
	A022-3B	TA01	T4203	Thu	1–2–3
	S020-3B	TA17	T3502	Thu	5–6–7
	C011-3B	TA11	T4302	Fri	2–3–4
IM4A	L020-4A	TA09	T3502	Tue	1–2–3
	S022-4A	TA05	T4302	Tue	6–7–8
	L017-4A	TA08	T3402	Wed	2–3–4
	S017-4A	TA17	T4302	Thu	1–2–3
	L021-4A	TA07	T4203	Thu	5–6–7
	T031-4A	TA15	T2202	Fri	6–7–8
IM4B	L015-4B	TA08	T4203	Tue	1–2–3
	T031-4B	TA15	T2202	Tue	5–6–7
	L020-4B	TA09	T4302	Wed	2–3–4
	S022-4B	TA12	T3402	Thu	6–7–8
	L018-4B	TA02	T4201	Fri	2–3–4
	D013-4B	TA07	T3502	Fri	6–7–8

(particle) represents the timetables of instructors that ensure that each timetable must be satisfied the hard constraints described in Section 3 and is mapped into class timetable and room timetable (see Fig. 3).

The next stage is to use the proposed algorithm to adjust the position of particles. This algorithm starts at the beginning of the first dimension and works through to the end of the particle. As mentioned in Section 4.2.3, the algorithm uses the new velocity based on Eq. (2) to update the particle current position to a new position according to Eq. (3). It is worth noting that the new position must be feasible in instructor's timetable (satisfy hard constraint 1), and the next step is to check whether the corresponding class timetables and room timetables satisfy the hard constraints 3 and 4, respectively. Fig. 7 shows the example, the first instructor teaches a course section CS1 with three consecutive time periods of 1, 2 and 3 on Monday for class 1. However, the next instructor also has a course section SE2 for class 1 at the same day and the same time. This means that the timetable of class 1 violates the hard constraint 3 and needs to be corrected. Pongcharoen et al. (2008) indicated that there are three approaches to deal with infeasible timetables: discard them, apply a high penalty in the fitness function and repair them. They showed that discarding infeasible solutions or applying a high penalty is only an option when a large proportion of the population is feasible.

This study adopts a repair process. The first step is to identify the instructors that clash (in this case CS1 and SE2). The process then determines the course section that one of the instructors is randomly selected for reallocation (say SE2). The second step is to regenerate new velocity of course section SE2 to identify a position (timeslot) for reallocation. The process then checks through the instructor's timetable, the class timetable and room timetable. If the new generated timeslot is feasible for all timetables, then the instructor timetable is to be updated. Otherwise, repeat step 2 until a timeslot is obtained that satisfies all hard constraints.

Table 5
Timetabling of various size problems.

Datasets	No. of Instructors	No. of classes
Small size	17	8
Medium size	32	16
Large size	49	28

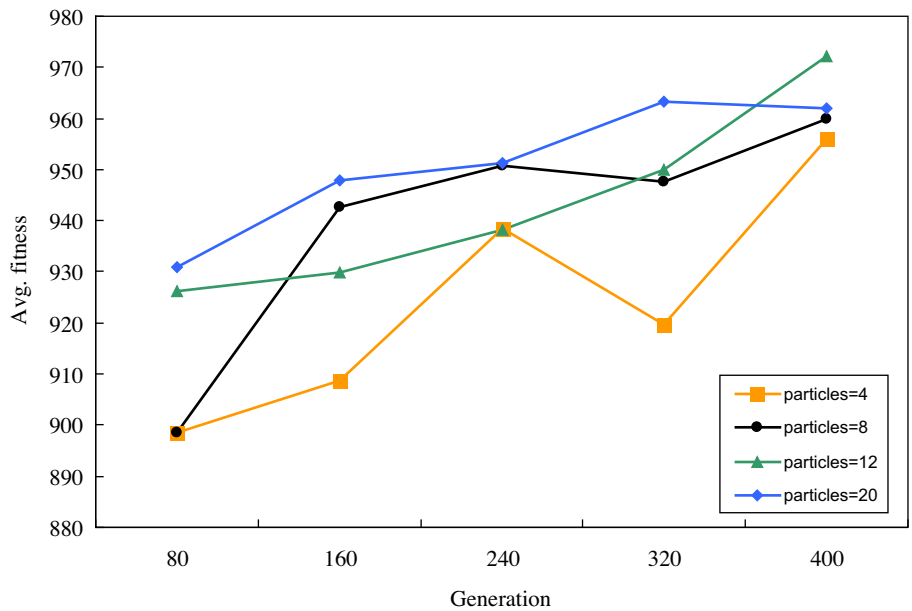

Fig. 10. The average fitness values for various numbers of particles.

4.2.6. Step of the proposed algorithm for UCSP

Fig. 8 illustrates the details of the proposed HPSO algorithm. The notations are used as follows:

Let P denotes the size of the PSO population.
Let p denotes the number of course sections offered by the problem.
Let m denotes the number of instructors to be scheduled.
Let $PSO[i]$ denotes the position of particle i of the PSO population represented as an p- dimensional vector, whose entries' values belong to the set $(1, 2, \ldots, 40 \times m)$.
Let $PSO[i][j]$ denotes the starting timeslot number to which the jth course section in the ith particle is assigned.
Let $fitness[i]$ denotes the fitness value of particle i based on Eq. (1).
Let $pbest[i]$ denotes the position of the local best position of particle i.
Let G_{best} denotes the index to global best position.
Let $pbest_fitness[i]$ denotes the local best fitness of particle i.
Let $V[i]$ denotes the velocity of particle i represented as an p-dimensional real coded vector.

5. Experimental results and discussion

5.1. Preliminary experiment

The HPSO approach described in Section 4 was implemented in visual C++ to conduct necessary experimentation (including a user friendly Graphical User Interface). The experiments are tested on a Pentium 4 – 3.40 GHz CPU with 512 MB RAM. The tested problems were performed using the data sets from the department of information management of Kun-Shan University in Taiwan, shown in Table 2. The performance of the PSO algorithm depends on the parameters chosen. The preliminary experiment used small size problem to identify the best configurations for PSO. To determine suitable parameter settings, four swarm sizes M (4, 8, 12, 20) were tested, where $M = 12$ was superior, and the parameters c_1 and c_2 were tested between 0.1 and 1 in increments of 0.1, as shown in Fig. 9. The settings $c_1 = 0.4$, $c_2 = 0.6$ were superior and used for all further studies, and the inertia weight w is decreased linearly from 0.9 to 0.2 during a run, V_{max} is set to 3 and L_{max} described in Section 4.2.4 is set to 10. In addition, α and β described in Eq. (1) were set to 0.8 and 0.2, respectively. Three other parameters are chosen as follows:

(1) Number of generations $GN = \{80, 160, 240, 320, 400\}$.
(2) Number of instructors $m = 17$.
(3) Number of classes $n = 8$.

5.2. Results of course scheduling for small size problem from HPSO

To illustrate the performance of the HPSO algorithm for UCSP ten runs were tested for each of GN. The experiments carried out that the most particles generated during each generation satisfy hard constraint 1. Since there are more time periods available to allocate those course sections lectured by each instructor. In addition, the new position of a course section generated during each generation has a higher probability to violate hard constraints 3, 4 and 5. However, they were always satisfied by repair process during each generation. As described earlier, it is usually difficult or impossible to generate course schedules that satisfy all instructors' preferences (Burke & Petrovic, 2002). According to the instructors' and classes' preferences shown in Appendixes A and B, it is worth mentioned that the results of course scheduling were achieved after 400 generations, which satisfy the all hard constraints and soft constraints 2 and 3 with a high satisfaction for instructors and classes, as shown in Tables 3 and 4.

5.3. Comparisons between GA and HPSO algorithm

To evaluate the HPSO algorithm, experiment was conducted to make a comparison with an existing GA based on the algorithm provided by Pongcharoen et al. (2008). To increase the sizes of test problems, this experiment also collected two sets of timetabling data from two other departments shown in Table 5. To get the average performance of the HPSO and GA algorithm, twenty runs on each problem instance were performed and the solution quality was averaged. This experiment also increased the swarm sizes M (30, 60 and 80). The result of $M = 60$ was best. A preliminary experiment shows that the same values of the HPSO parameters

Table 6
Comparison with GA algorithm for various size problems.

Problem size $m \times n$	Improvement in APD(%)	HPSO		GA		CT(sec)	
		SCV2	SCV3	SCV2	SCV3	HPSO	GA
17×8	+2.45	0.95	0.13	2.28	2.34	3.85	58.95
32×16	+6.78	2.16	2.05	6.41	7.23	9.13	108.27
49×28	−0.58	5.69	3.28	12.51	8.54	25.08	213.26
Average	+2.88	2.93	1.82	7.06	6.03	12.68	126.82

as in Section 5.1 are also suitable here. According to the best solutions were obtained in Pongcharoen et al. (2008), they suggested that the size of population and the number of generation of GA algorithm were set to 500 and 100, respectively. Both algorithms used the same fitness function described in Eq. (1) to evaluate the population. The percentage deviation (PD) in TPV (fitness) is used as the performance measure, i.e., PD = $(TPV_{HPSO} \times TPV_{GA}) / TPV_{GA} \times 100$, where TPV_{HPSO} represents the best total priority values (TPV) of each run generated by HPSO algorithm and TPV_{GA} represents the best total priority values (TPV) generated by GA algorithm. For each instance, the average PD (APD) of 20 trials is computed. The results are given in Table 6, which gives the improvement in APD, the average number of times that violates the soft constraint 2 (SCV2), the average number of times that violates the soft constraint 3 (SCV3) and mean computation time (CT) in seconds.

The experimental solution shows clearly that the fitness value (TPV) obtained by HPSO algorithm is better than the GA for the problem of small and medium sizes. In term of large size in TPV, HPSO was slightly worse than that of GA. However, the HPSO obtained lower SCV2 and SCV3 than the GA for all sizes of problems. The GA approach tends to generate infeasible timetables due to the crossover and the mutation operators. i.e., the most chromosomes violated hard constraints 1, 4 and 6 that the large number of repairs is necessary and more CPU time is required. Compared with GA, the advantages of HPSO algorithm are that HPSO is easy to implement and there are fewer parameters to adjust.

Experimental results demonstrate some significant consequences and the features of PSO when applied to UCSP examined herein, as follows.

(1) The parameters of PSO search method are less than other meta-heuristics. It is observed from the experimental results that the extreme parameter values possibly are unable to obtain the better solutions. Instead, the experiments suggest that an intermediate value is chosen from the relevant parameters (see Fig. 9).
(2) According to the experimental results, the excessive number of particles did not necessary obtain the higher fitness values; However, less number of particles involves unstable solutions, as shown in Fig. 10.
(3) Regarding the number of generations during the PSO process, an average of 200 generations is needed to converge to a better and stable solution.

6. Conclusions

This study has investigated the HPSO approach in a course scheduling problem. The proposed HPSO algorithm contains several features. The first is to design an 'absolute position value' representation for the particle. The second is to allow instructors that they are willing to lecture based on flexible preferences, such as their preferred days and time periods, the maximum number of teaching-free time periods and the lecturing format (consecutive time periods or separated into different time periods). The third, a repair process is involved, which ensures that all infeasible timetables are rectified. Moreover, since the solution space of UCSP is discrete, and a local search mechanism is incorporated to explore a better solution improvement.

To the best of knowledge, this is the first attempt to incorporate the flexibilities of soft constraints (described in section 3) into the fitness function simultaneously. Previous research has demonstrated their algorithms that only consider the parts of four soft constraints. However, those flexible preferences are considered to be important by many academics. The experimental results demonstrate that the timetables obtained from the proposed approach yields an optimal satisfaction for instructors and students (classes) according to their preferences. Further research encourages that the extension of PSO incorporating other heuristic maybe is more effective for UCSP. In summary, the results proposed in this study are very encouraging and promising for the application of the PSO algorithm to UCSP and other scheduling problems.

Acknowledgements

This work was supported by the National Science Council, Taiwan, Republic of China, under grant number NSC 99-2221-E-242 -006.

Appendix A. The priority values of preference settings for instructors

Period	TA01					TA02				
	Mon	Tue	Wed	Thu	Fri	Mon	Tue	Wed	Thu	Fri
1	−1	4	5	2	5	5	0	5	−1	4
2	−1	4	5	2	5	5	1	5	−1	4
3	−1	4	5	2	5	5	1	5	−1	4
4	−1	4	5	2	5	5	1	5	−1	4
5	5	1	−1	1	4	−1	4	−1	5	1
6	5	1	−1	1	4	−1	4	−1	5	1
7	5	1	−1	1	4	−1	4	−1	5	1
8	0	1	−1	1	0	−1	0	−1	0	0

(*continued on next page*)

Appendix A (*continued*)

Period	TA03					TA04				
	Mon	Tue	Wed	Thu	Fri	Mon	Tue	Wed	Thu	Fri
1	5	−1	5	2	5	1	1	5	−1	4
2	5	−1	5	5	5	1	1	5	−1	4
3	5	−1	5	5	5	1	1	5	−1	4
4	5	−1	5	5	5	1	1	5	−1	0
5	−1	3	−1	5	5	5	4	−1	5	1
6	−1	3	−1	5	5	5	4	−1	5	1
7	−1	3	−1	5	5	5	4	−1	5	1
8	0	0	−1	0	0	0	0	−1	0	0
	TA05					TA06				
1	0	1	1	2	4	5	4	5	−1	5
2	4	1	5	2	4	5	4	5	3	5
3	4	1	5	2	4	5	4	5	3	5
4	4	1	5	2	4	5	4	5	3	5
5	1	5	−1	3	2	0	1	−1	3	0
6	1	5	−1	3	2	0	1	−1	3	0
7	1	5	−1	3	2	0	1	−1	3	0
8	1	0	−1	0	0	0	0	−1	0	0
	TA07					TA08				
1	3	−1	5	−1	3	−1	4	5	1	3
2	3	−1	5	−1	3	−1	4	5	1	3
3	3	−1	5	−1	3	−1	4	5	1	3
4	0	−1	5	−1	3	−1	4	5	1	3
5	3	5	−1	5	3	5	2	−1	5	2
6	3	5	−1	5	3	5	2	−1	5	2
7	3	5	−1	5	3	5	1	−1	5	2
8	0	1	−1	2	0	0	1	−1	0	0
	TA09					TA10				
1	3	4	5	5	0	−1	2	5	3	4
2	3	4	5	5	0	5	2	5	3	4
3	3	4	5	5	0	5	2	5	3	4
4	3	0	5	5	0	5	2	5	0	4
5	3	1	−1	−1	5	−1	4	−1	3	1
6	3	1	−1	−1	5	−1	4	−1	3	1
7	3	1	−1	−1	5	−1	4	−1	3	1
8	0	1	−1	−1	0	−1	0	−1	3	1
	TA11					TA12				
1	0	3	5	4	0	4	5	5	2	3
2	0	3	5	4	0	4	5	5	2	3
3	0	3	5	4	0	4	5	5	2	3
4	0	3	5	4	0	0	5	5	2	3
5	5	3	−1	1	5	2	0	−1	3	3
6	5	3	−1	1	5	2	0	−1	3	3
7	5	3	−1	1	5	2	0	−1	3	3
8	5	0	−1	1	0	0	0	−1	0	0
	TA13					TA14				
1	0	2	5	4	5	4	1	5	5	5
2	1	2	5	4	5	4	1	5	5	5
3	1	2	5	4	5	4	1	5	5	5
4	1	2	5	4	5	4	1	5	5	5
5	5	3	−1	1	1	2	4	−1	1	1
6	5	3	−1	1	1	2	4	−1	1	1
7	5	3	−1	1	1	2	4	−1	1	1
8	0	0	−1	1	1	−1	0	−1	1	1

Appendix A (*continued*)

Period	TA15					TA16				
	Mon	Tue	Wed	Thu	Fri	Mon	Tue	Wed	Thu	Fri
1	3	0	5	0	0	2	5	5	4	0
2	3	0	5	0	0	2	5	5	4	0
3	3	0	5	0	0	2	5	5	4	0
4	3	0	5	0	0	0	5	5	4	0
5	3	5	−1	5	5	3	−1	−1	1	5
6	3	5	−1	5	5	3	−1	−1	1	5
7	3	5	−1	5	5	3	−1	−1	1	5
8	0	5	−1	0	0	0	−1	−1	1	0

Period	TA17				
	Mon	Tue	Wed	Thu	Fri
1	4	2	5	4	5
2	4	2	5	4	5
3	4	2	5	4	5
4	4	2	5	4	5
5	1	3	−1	2	−1
6	1	3	−1	2	−1
7	1	3	−1	2	−1
8	1	0	−1	0	−1

Appendix B. The priority values of preference settings for classes

Period	Class IM1A					Class IM1B				
	Mon	Tue	Wed	Thu	Fri	Mon	Tue	Wed	Thu	Fri
1	0	0	0	0	0	0	0	0	0	0
2	1	3	2	3	2	2	3	5	4	4
3	1	3	2	3	2	2	3	5	4	4
4	1	3	2	3	2	2	3	5	4	4
5	4	4	−1	4	0	3	3	−1	2	0
6	4	4	−1	4	0	3	3	−1	2	0
7	0	4	−1	4	0	3	3	−1	2	0
8	0	0	−1	0	0	0	0	−1	0	0
	Class IM2A					Class IM2B				
1	0	0	5	0	0	0	0	0	0	0
2	0	4	5	5	5	0	3	5	3	3
3	0	4	5	5	5	0	3	5	3	3
4	0	4	5	5	5	0	3	5	3	3
5	5	2	−1	2	0	5	4	−1	1	3
6	5	2	−1	2	0	5	4	−1	1	3
7	5	2	−1	2	0	5	4	−1	1	3
8	0	0	−1	0	0	0	0	−1	0	0
	Class IM3A					Class IM3B				
1	0	0	0	0	4	3	0	5	4	5
2	3	2	5	2	4	3	4	5	4	5
3	3	2	5	2	4	3	4	5	4	5
4	3	2	5	2	4	3	4	5	4	5
5	3	4	−1	4	1	2	2	−1	1	0
6	3	4	−1	4	1	2	2	−1	1	0
7	3	4	−1	4	1	2	2	−1	1	0
8	0	0	−1	0	0	0	0	−1	0	0

(*continued on next page*)

Appendix B (*continued*)

Period	Class IM4A					Class IM4B				
	Mon	Tue	Wed	Thu	Fri	Mon	Tue	Wed	Thu	Fri
1	0	1	0	5	0	0	5	0	0	0
2	3	4	5	5	5	0	5	5	3	4
3	3	4	5	5	5	0	5	5	3	4
4	3	4	5	5	5	0	5	5	3	4
5	2	2	−1	1	1	5	0	−1	3	1
6	2	2	−1	1	1	5	0	−1	3	1
7	2	2	−1	1	1	5	0	−1	3	1
8	0	0	−1	0	0	0	0	−1	0	0

References

Alvarez-Valdes, R., Crespo, E., & Tamarit, J. M. (2002). Design and implementation of a course scheduling system using tabu search. *European Journal of Operational Research, 137*(3), 512–523.

Azimi, Z. N. (2005). Hybrid heuristics for examination timetabling problem. *Applied Mathematics and Computation, 163*(2), 705–733.

Bardadym, V. A. (1996). Computer-aided school and university timetabling: The new wave. In E. Burke & P. Ross (Eds.), *Practice and theory of automated timetabling. Lecture notes in computer science* (Vol. 1153, pp. 22–45). Berlin: Springer.

Burke, E. K., & Petrovic, S. (2002). Recent research directions in automated timetabling. *European Journal of Operational Research, 140*(2), 266–280.

Chiarandini, M., Birattari, M., Socha, K., & Rossi-Doria, O. (2006). An effective hybrid algorithm for university course timetabling. *Journal of Scheduling, 9*(5), 403–432.

Chiarandini, M., & Stützle, T. (2003). *A landscape analysis for a hybrid approximate algorithm on a timetabling problem*. TU Darmstadt Technical Report AIDA-03-05.

Combs, W., Hawkins, R., Pore, T., Schechet, A., Wahls, T., & Ziantz, L. (2005). The course scheduling problem as a source of student projects. In *Proceedings of the 36th SIGCSE technical symposium on computer science education SIGCSE'05* (pp. 81–85).

Drexl, A., & Salewski, F. (1997). Distribution requirements and compactness constraints in school timetabling. *European Journal of Operational Research, 102*(1), 193–214.

Duong, T. A., & Lam, K. H. (2004). Combining constraint programming and simulated annealing on university exam timetabling. In *Proceedings of international conference RIVF'04, Hanoi* (pp. 205–210).

Gunawan, A., Ng, J. M., & Poh, K. L. (2007). Solving the teacher assignment-course scheduling problem by a hybrid algorithm. *International Journal of Computer, Information, and System Science, and Engineering, 1*(2), 136–141.

Kennedy, J., & Eberhart, R. C. (1995). Particle swarm optimization. In *Proceedings of IEEE international conference on neural networks* (pp. 1942–1948).

Kuo, I. H., Horng, S. J., Kao, T. W., Lin, T. L., Lee, C. L., Terano, T., et al. (2009). An efficient flow-shop scheduling algorithm based on a hybrid particle swarm optimization model. *Expert Systems with Applications, 36*(3), 7027–7032.

Lo, S. T, Chen, R. M., Shiau, D. F., & Wu, C. L. (2008). Using particle swarm optimization to solve resource-constrained scheduling problems. In *IEEE conference on soft computing in industrial applications* (pp. 38–43).

Luo, X., Wang, D., Tang, J., & Tu, Y. (2006). An improved PSO algorithm for resource-constrained project scheduling problem. In *The 6th world congress on intelligent control and automation, WCICA 2006* (Vol. 1, pp. 3514–3518).

Marriott, K., & Stuckey, P. J. (1998). *Programming with constraints: An Introduction*. Cambridge, Massachusetts: The MIT Press.

Merlot, L. T. G., Boland, N., Hughes, B. D., & Stuckey, P. J. (2002). A hybrid algorithm for the examination timetabling problem. In E. Burke & M. Carter (Eds.), *Practice and theory of automated timetabling IV. Lecture notes in computer science* (Vol. 2740, pp. 207–231). New York: Springer-Verlag.

Pongcharoen, P., Promtet, W., Yenradee, P., & Hicks, C. (2008). Stochastic optimisation timetabling tool for university course scheduling. *International Journal of Production Economics, 112*(2), 903–918.

Salman, A., Ahmad, I., & Al-Madani, S. (2002). Particle swarm optimization for task assignment problem. *Microprocessors and Microsystems, 26*, 363–371.

Sha, D. Y., & Hsu, C. Y. (2006). A hybrid particle swarm optimization for job shop scheduling problem. *Computers and Industrial Engineering, 51*, 791–808.

Shiau, D. F., Cheng, S. C., & Huang, Y. M. (2008). Proportionate flexible flow shop scheduling via a hybrid constructive genetic algorithm. *Expert Systems with Applications, 34*, 1133–1143.

Thompson, J. M., & Dowsland, K. A. (1998). A robust simulated annealing based examination timetabling system. *Computers and Operations Research, 25*(7-8), 637–648.

Tseng, C. T., & Liao, C. J. (2008). A particle swarm optimization algorithm for hybrid flow-shop scheduling with multiprocessor tasks. *International Journal of Production Research, 46*(17), 4655–4670.

Valdes, R. A., Crespo, E., & Tamarit, J. M. (2002). Design and implementation of a course scheduling system using tabu Search. *European Journal of Operational Research, 137*, 512–523.

Wang, Y. Z. (2002). An application of genetic algorithm methods for teacher assignment problems. *Expert Systems with Applications, 22*(4), 295–302.

Wang, Y. Z. (2003). Using genetic algorithm methods to solve course scheduling problems. *Expert Systems with Applications, 25*(1), 39–50.

Wang, H. (2005). Flexible flow shop scheduling: Optimum, heuristics and artificial intelligence solutions. *Expert Systems, 22*(2), 78–85.

Weare, R., Burke, E., & Elliman, D. (1995). *A hybrid genetic algorithm for examination timetabling problem*. University of Nottingham, Computer Science Technical Report No. NOTTCS-TR-1995-8.

Zhang, H., Li, H., & Tam, C. M. (2006). Particle swarm optimization for resource-constrained project scheduling. *International Journal of Project Management, 24*(1), 83–92.